外部ライブラリを使わず、単1シートの Open XML (.xlsx) を生成する

表紙

項目	内容
文書名	基本設計書
システム名	
機能ID	COM-150
画面名	外部ライブラリを使わず、単1シートの Open XML (.xlsx) を生成する
会社名	
作成日	

概要

概要説明

外部ライブラリを使わず、単1シートの Open XML (.xlsx) を生成するの入力・表示・要求処理を行う。

情報	内容
対象種別	common

情報	内容
開始契機	呼出元からの要求
処理対象	入力値・画面状態

項目一覧

No	グループ	項目名	種類	入出力	DB連携項目	候補取得元	必須	形式・桁	説明

項目定義数：0項目。表示順と反復行の定義数を区別する。

イベント仕様

OpenXmlWorkbookWriter

処理概要

外部ライブラリを使わず、単1シートの Open XML (.xlsx) を生成するで「OpenXmlWorkbookWriter」の要求を受け付ける。入力・確認・データ操作・応答の順と、条件が成立しない場合の処理を以下に示す。

処理手順

1. 関連する処理へ制御を渡す。詳細な実行経路は原本対応を参照する。

buildWorkbook

処理概要

外部ライブラリを使わず、単1シートの Open XML (.xlsx) を生成するで「buildWorkbook」の要求を受け付ける。入力・確認・データ操作・応答の順と、条件が成立しない場合の処理を以下に示す。

処理手順

1. 処理結果を呼出元へ返す。この経路の後続手順は実行しない。

結果・例外時の扱い

到達条件	結果
この終了経路へ到達した場合	処理結果を呼出元へ返す

buildWorkbook

処理概要

外部ライブラリを使わず、単1シートの Open XML (.xlsx) を生成するで「buildWorkbook」の要求を受け付ける。入力・確認・データ操作・応答の順と、条件が成立しない場合の処理を以下に示す。

処理手順

1. 「対象項目がない、または「対象項目」」に該当するか確認する。該当する場合に限り、次の分岐内手順を行う。該当しない場合は分岐の後へ進む。

1.1. 条件成立時：処理の異常を通知し、この経路を終了する。この経路の後続手順は実行しない。

    1. 「対象項目がない」に該当するか確認する。該当する場合に限り、次の分岐内手順を行う。該当しない場合は分岐の後へ進む。

2.1. 条件成立時：処理の異常を通知し、この経路を終了する。この経路の後続手順は実行しない。

    1. 関連情報を取得・設定する（処理10：詳細仕様参照）。
    2. 関連情報を取得・設定する（処理11：詳細仕様参照）。
    3. 通常処理と例外時の処理を区別して実行する。例外処理は対応する例外が発生した場合に限る。

5.1. 通常処理：関連情報を取得・設定する（処理12：詳細仕様参照）。

5.2. 通常処理：関連情報を取得・設定する（処理13：詳細仕様参照）。

5.3. 通常処理：関連情報を取得・設定する（処理14：詳細仕様参照）。

5.4. 通常処理：関連情報を取得・設定する（処理15：詳細仕様参照）。

5.5. 通常処理：関連情報を取得・設定する（処理16：詳細仕様参照）。

5.6. 通常処理：関連情報を取得・設定する（処理17：詳細仕様参照）。

5.7. 通常処理：関連情報を取得・設定する（処理18：詳細仕様参照）。

5.8. 通常処理：終了時の共通後処理を行ってから、処理結果を呼出元へ返す。この経路の後続手順は実行しない。

5-例外1. 例外条件「IOException e」に該当した場合に次の手順を行う。型名は原本の表記。

5-例外1.1. 例外発生時：終了時の共通後処理を行ってから、処理の異常を通知し、この経路を終了する。この経路の後続手順は実行しない。

5-後処理. 通常終了・例外・途中のreturnのいずれでも、この範囲を抜ける前に次の共通後処理を行う。

5-後処理.1. 終了時の共通後処理：通常処理と例外時の処理を区別して実行する。例外処理は対応する例外が発生した場合に限る。

5-後処理.1.1. 通常処理：使用した接続・処理資源を解放する。

5-後処理.1-例外1. 例外条件「IOException e」に該当した場合に次の手順を行う。型名は原本の表記。

結果・例外時の扱い

到達条件	結果
対象項目がない、または「対象項目」	処理の異常を通知し、この経路を終了する
NOT (headers が等しい 未設定 || 「対象項目」)・対象項目がない	処理の異常を通知し、この経路を終了する
NOT (headers が等しい 未設定 || 「対象項目」)・NOT (rows が等しい 未設定)	処理結果を呼出元へ返す
NOT (headers が等しい 未設定 || 「対象項目」)・NOT (rows が等しい 未設定)・条件19を満たす（原本条件は補足説明を参照）	処理の異常を通知し、この経路を終了する

contentTypes

処理概要

外部ライブラリを使わず、単1シートの Open XML (.xlsx) を生成するで「contentTypes」の要求を受け付ける。入力・確認・データ操作・応答の順と、条件が成立しない場合の処理を以下に示す。

処理手順

1. 処理結果を呼出元へ返す。この経路の後続手順は実行しない。

結果・例外時の扱い

到達条件	結果
この終了経路へ到達した場合	処理結果を呼出元へ返す

rootRelationships

処理概要

外部ライブラリを使わず、単1シートの Open XML (.xlsx) を生成するで「rootRelationships」の要求を受け付ける。入力・確認・データ操作・応答の順と、条件が成立しない場合の処理を以下に示す。

処理手順

1. 処理結果を呼出元へ返す。この経路の後続手順は実行しない。

結果・例外時の扱い

到達条件	結果
この終了経路へ到達した場合	処理結果を呼出元へ返す

workbookXml

処理概要

外部ライブラリを使わず、単1シートの Open XML (.xlsx) を生成するで「workbookXml」の要求を受け付ける。入力・確認・データ操作・応答の順と、条件が成立しない場合の処理を以下に示す。

処理手順

1. 処理結果を呼出元へ返す。この経路の後続手順は実行しない。

結果・例外時の扱い

到達条件	結果
この終了経路へ到達した場合	処理結果を呼出元へ返す

workbookRelationships

処理概要

外部ライブラリを使わず、単1シートの Open XML (.xlsx) を生成するで「workbookRelationships」の要求を受け付ける。入力・確認・データ操作・応答の順と、条件が成立しない場合の処理を以下に示す。

処理手順

1. 処理結果を呼出元へ返す。この経路の後続手順は実行しない。

結果・例外時の扱い

到達条件	結果
この終了経路へ到達した場合	処理結果を呼出元へ返す

stylesXml

処理概要

外部ライブラリを使わず、単1シートの Open XML (.xlsx) を生成するで「stylesXml」の要求を受け付ける。入力・確認・データ操作・応答の順と、条件が成立しない場合の処理を以下に示す。

処理手順

1. エラーまたは処理結果「<?xml version="1.0" encoding="UTF-8" standalone="yes"?>" + "<styleSheet xmlns="" + SPREADSHEET_NS + "">" + "<fonts count="4"><sz val="10"/><name val="メイリオ"/>" + "<b/><sz val="10"/><name val="メイリオ"/>" + "<b/><sz val="22"/><name val="メイリオ"/>" + "<sz val="16"/><name val="メイリオ"/></fonts>" + "<fills count="3"><fill><patternFill patternType="none"/></fill>" + "<fill><patternFill patternType="gray125"/></fill>" + "<fill><patternFill patternType="solid"><fgColor rgb="FFFFFF96"/>" + "<bgColor indexed="64"/></patternFill></fill></fills>" + "<borders count="2"><border><left/><right/><top/><bottom/><diagonal/></border>" + "<border><left style="thin"/><right style="thin"/><top style="thin"/>" + "<bottom style="thin"/><diagonal/></border></borders>" + "<cellStyleXfs count="1"><xf numFmtId="0" fontId="0" fillId="0" borderId="0"/></cellStyleXfs>" + "<cellXfs count="5">" + "<xf numFmtId="0" fontId="0" fillId="0" borderId="1" xfId="0" applyBorder="1"/>" + "<xf numFmtId="0" fontId="1" fillId="2" borderId="1" xfId="0" applyFont="1" applyFill="1" applyBorder="1" applyAlignment="1"><alignment horizontal="center" vertical="center" wrapText="1"/></xf>" + "<xf numFmtId="0" fontId="0" fillId="0" borderId="1" xfId="0" applyBorder="1" applyAlignment="1"><alignment horizontal="right" vertical="top"/></xf>" + "<xf numFmtId="0" fontId="2" fillId="0" borderId="0" xfId="0" applyFont="1" applyAlignment="1"><alignment horizontal="center" vertical="center"/></xf>" + "<xf numFmtId="0" fontId="3" fillId="0" borderId="0" xfId="0" applyFont="1" applyAlignment="1"><alignment horizontal="center" vertical="center"/></xf>" + "</cellXfs><cellStyles count="1"><cellStyle name="Normal" xfId="0" builtinId="0"/></cellStyles>" + "</styleSheet>」を返す。この経路の後続手順は実行しない。

結果・例外時の扱い

到達条件	結果
この終了経路へ到達した場合	エラーまたは処理結果「<?xml version="1.0" encoding="UTF-8" standalone="yes"?>" + "<styleSheet xmlns="" + SPREADSHEET_NS + ""><fonts count="4"><sz val="10"/><name val="メイリオ"/>" + "<b/><sz val="10"/><name val="メイリオ"/>" + "<b/><sz val="22"/><name val="メイリオ"/>" + "<sz val="16"/><name val="メイリオ"/></fonts>" + "<fills count="3"><fill><patternFill patternType="none"/></fill>" + "<fill><patternFill patternType="gray125"/></fill>" + "<fill><patternFill patternType="solid"><fgColor rgb="FFFFFF96"/>" + "<bgColor indexed="64"/></patternFill></fill></fills>" + "<borders count="2"><border><left/><right/><top/><bottom/><diagonal/></border>" + "<border><left style="thin"/><right style="thin"/><top style="thin"/>" + "<bottom style="thin"/><diagonal/></border></borders>" + "<cellStyleXfs count="1"><xf numFmtId="0" fontId="0" fillId="0" borderId="0"/></cellStyleXfs>" + "<cellXfs count="5">" + "<xf numFmtId="0" fontId="0" fillId="0" borderId="1" xfId="0" applyBorder="1"/>" + "<xf numFmtId="0" fontId="1" fillId="2" borderId="1" xfId="0" applyFont="1" applyFill="1" applyBorder="1" applyAlignment="1"><alignment horizontal="center" vertical="center" wrapText="1"/></xf>" + "<xf numFmtId="0" fontId="0" fillId="0" borderId="1" xfId="0" applyBorder="1" applyAlignment="1"><alignment horizontal="right" vertical="top"/></xf>" + "<xf numFmtId="0" fontId="2" fillId="0" borderId="0" xfId="0" applyFont="1" applyAlignment="1"><alignment horizontal="center" vertical="center"/></xf>" + "<xf numFmtId="0" fontId="3" fillId="0" borderId="0" xfId="0" applyFont="1" applyAlignment="1"><alignment horizontal="center" vertical="center"/></xf>" + "</cellXfs><cellStyles count="1"><cellStyle name="Normal" xfId="0" builtinId="0"/></cellStyles>" + "</styleSheet>」を返す

sheetXml

処理概要

外部ライブラリを使わず、単1シートの Open XML (.xlsx) を生成するで「sheetXml」の要求を受け付ける。入力・確認・データ操作・応答の順と、条件が成立しない場合の処理を以下に示す。

処理手順

1. 関連情報を取得・設定する（処理43：詳細仕様参照）。
2. 関連情報を取得・設定する（処理44：詳細仕様参照）。
3. 関連情報を取得・設定する（処理45：詳細仕様参照）。
4. 関連情報を取得・設定する（処理46：詳細仕様参照）。
5. 関連情報を取得・設定する（処理47：詳細仕様参照）。
6. 関連情報を取得・設定する（処理48：詳細仕様参照）。
7. 関連情報を取得・設定する（処理49：詳細仕様参照）。
8. 関連情報を取得・設定する（処理50：詳細仕様参照）。
9. 関連情報を取得・設定する（処理51：詳細仕様参照）。
10. 関連情報を取得・設定する（処理52：詳細仕様参照）。
11. 関連情報を取得・設定する（処理53：詳細仕様参照）。
12. 「対象項目がある」に該当するか確認する。該当する場合に限り、次の分岐内手順を行う。該当しない場合は分岐の後へ進む。

12.1. 条件成立時：繰返し条件「条件54を満たす（原本条件は補足説明を参照）」に従い、次の手順を繰り返す。条件を満たさなくなった場合は繰返しの後へ進む。

12.1.1. 繰返しの各回：関連情報を取得・設定する（処理55：詳細仕様参照）。

12.1.2. 繰返しの各回：関連情報を取得・設定する（処理56：詳細仕様参照）。

12.1.3. 繰返しの各回：関連情報を取得・設定する（処理57：詳細仕様参照）。

12.1.4. 繰返しの各回：「excelRow が等しい 1」に該当するか確認する。該当する場合に限り、次の分岐内手順を行う。該当しない場合は分岐の後へ進む。

12.1.4.1. 条件成立時：関連情報を取得・設定する（処理58：詳細仕様参照）。

12.1.5. 繰返しの各回：関連情報を取得・設定する（処理59：詳細仕様参照）。

12.1.6. 繰返しの各回：「対象項目がある」に該当するか確認する。該当する場合に限り、次の分岐内手順を行う。該当しない場合は分岐の後へ進む。

12.1.6.1. 条件成立時：繰返し条件「条件60を満たす（原本条件は補足説明を参照）」に従い、次の手順を繰り返す。条件を満たさなくなった場合は繰返しの後へ進む。

12.1.6.1.1. 繰返しの各回：関連情報を取得・設定する（処理61：詳細仕様参照）。

12.1.6.1.2. 繰返しの各回：「対象項目がある」に該当するか確認する。該当する場合に限り、次の分岐内手順を行う。該当しない場合は分岐の後へ進む。

12.1.6.1.2.1. 条件成立時：関連情報を取得・設定する（処理62：詳細仕様参照）。

12.1.7. 繰返しの各回：関連情報を取得・設定する（処理63：詳細仕様参照）。

1. 関連情報を取得・設定する（処理64：詳細仕様参照）。
2. 繰返し条件「条件65を満たす（原本条件は補足説明を参照）」に従い、次の手順を繰り返す。条件を満たさなくなった場合は繰返しの後へ進む。

14.1. 繰返しの各回：関連情報を取得・設定する（処理66：詳細仕様参照）。

1. 関連情報を取得・設定する（処理67：詳細仕様参照）。
2. 繰返し条件「条件68を満たす（原本条件は補足説明を参照）」に従い、次の手順を繰り返す。条件を満たさなくなった場合は繰返しの後へ進む。

16.1. 繰返しの各回：関連情報を取得・設定する（処理69：詳細仕様参照）。

16.2. 繰返しの各回：関連情報を取得・設定する（処理70：詳細仕様参照）。

16.3. 繰返しの各回：関連情報を取得・設定する（処理71：詳細仕様参照）。

16.4. 繰返しの各回：繰返し条件「条件72を満たす（原本条件は補足説明を参照）」に従い、次の手順を繰り返す。条件を満たさなくなった場合は繰返しの後へ進む。

16.4.1. 繰返しの各回：関連情報を取得・設定する（処理73：詳細仕様参照）。

16.4.2. 繰返しの各回：関連情報を取得・設定する（処理74：詳細仕様参照）。

16.4.3. 繰返しの各回：「条件75を満たす（原本条件は補足説明を参照）」に該当するか確認する。成立時と不成立時の手順を分けて行う。

16.4.3.1. 条件成立時：関連情報を取得・設定する（処理76：詳細仕様参照）。

16.4.3-別.1. 条件不成立時：関連情報を取得・設定する（処理77：詳細仕様参照）。

16.5. 繰返しの各回：関連情報を取得・設定する（処理78：詳細仕様参照）。

1. 関連情報を取得・設定する（処理79：詳細仕様参照）。
2. 「対象項目がある、かつ!「対象項目」」に該当するか確認する。該当する場合に限り、次の分岐内手順を行う。該当しない場合は分岐の後へ進む。

18.1. 条件成立時：関連情報を取得・設定する（処理80：詳細仕様参照）。

18.2. 条件成立時：繰返し条件「条件81を満たす（原本条件は補足説明を参照）」に従い、次の手順を繰り返す。条件を満たさなくなった場合は繰返しの後へ進む。

18.2.1. 繰返しの各回：関連情報を取得・設定する（処理82：詳細仕様参照）。

18.3. 条件成立時：関連情報を取得・設定する（処理83：詳細仕様参照）。

1. 関連情報を取得・設定する（処理84：詳細仕様参照）。
2. 処理結果を呼出元へ返す。この経路の後続手順は実行しない。

結果・例外時の扱い

到達条件	結果
この終了経路へ到達した場合	処理結果を呼出元へ返す

appendColumns

処理概要

外部ライブラリを使わず、単1シートの Open XML (.xlsx) を生成するで「appendColumns」の要求を受け付ける。入力・確認・データ操作・応答の順と、条件が成立しない場合の処理を以下に示す。

処理手順

1. 関連情報を取得・設定する（処理12：詳細仕様参照）。
2. 繰返し条件「条件13を満たす（原本条件は補足説明を参照）」に従い、次の手順を繰り返す。条件を満たさなくなった場合は繰返しの後へ進む。

2.1. 繰返しの各回：関連情報を取得・設定する（処理14：詳細仕様参照）。

2.2. 繰返しの各回：関連情報を取得・設定する（処理15：詳細仕様参照）。

2.3. 繰返しの各回：繰返し条件「条件16を満たす（原本条件は補足説明を参照）」に従い、次の手順を繰り返す。条件を満たさなくなった場合は繰返しの後へ進む。

2.3.1. 繰返しの各回：関連情報を取得・設定する（処理17：詳細仕様参照）。

2.3.2. 繰返しの各回：関連情報を取得・設定する（処理18：詳細仕様参照）。

2.3.3. 繰返しの各回：関連情報を取得・設定する（処理19：詳細仕様参照）。

2.4. 繰返しの各回：関連情報を取得・設定する（処理20：詳細仕様参照）。

2.5. 繰返しの各回：関連情報を取得・設定する（処理21：詳細仕様参照）。

1. 関連情報を取得・設定する（処理22：詳細仕様参照）。

appendTextCell

処理概要

外部ライブラリを使わず、単1シートの Open XML (.xlsx) を生成するで「appendTextCell」の要求を受け付ける。入力・確認・データ操作・応答の順と、条件が成立しない場合の処理を以下に示す。

処理手順

1. 関連情報を取得・設定する（処理7：詳細仕様参照）。
2. 関連情報を取得・設定する（処理8：詳細仕様参照）。
3. 「条件9を満たす（原本条件は補足説明を参照）、かつ条件10を満たす（原本条件は補足説明を参照）」に該当するか確認する。該当する場合に限り、次の分岐内手順を行う。該当しない場合は分岐の後へ進む。

3.1. 条件成立時：関連情報を取得・設定する（処理11：詳細仕様参照）。

1. 関連情報を取得・設定する（処理12：詳細仕様参照）。

appendNumberCell

処理概要

外部ライブラリを使わず、単1シートの Open XML (.xlsx) を生成するで「appendNumberCell」の要求を受け付ける。入力・確認・データ操作・応答の順と、条件が成立しない場合の処理を以下に示す。

処理手順

1. 関連情報を取得・設定する（処理2：詳細仕様参照）。

displayLength

処理概要

外部ライブラリを使わず、単1シートの Open XML (.xlsx) を生成するで「displayLength」の要求を受け付ける。入力・確認・データ操作・応答の順と、条件が成立しない場合の処理を以下に示す。

処理手順

1. 関連情報を取得・設定する（処理6：詳細仕様参照）。
2. 繰返し条件「条件7を満たす（原本条件は補足説明を参照）」に従い、次の手順を繰り返す。条件を満たさなくなった場合は繰返しの後へ進む。

2.1. 繰返しの各回：関連情報を取得・設定する（処理8：詳細仕様参照）。

2.2. 繰返しの各回：関連情報を取得・設定する（処理9：詳細仕様参照）。

2.3. 繰返しの各回：関連情報を取得・設定する（処理10：詳細仕様参照）。

1. 処理結果を呼出元へ返す。この経路の後続手順は実行しない。

結果・例外時の扱い

到達条件	結果
この終了経路へ到達した場合	処理結果を呼出元へ返す

columnName

処理概要

外部ライブラリを使わず、単1シートの Open XML (.xlsx) を生成するで「columnName」の要求を受け付ける。入力・確認・データ操作・応答の順と、条件が成立しない場合の処理を以下に示す。

処理手順

1. 関連情報を取得・設定する（処理6：詳細仕様参照）。
2. 関連情報を取得・設定する（処理7：詳細仕様参照）。

3. 繰返し条件「条件8を満たす（原本条件は補足説明を参照）」に従い、次の手順を繰り返す。条件を満たさなくなった場合は繰返しの後へ進む。

3.1. 繰返しの各回：関連情報を取得・設定する（処理9：詳細仕様参照）。

3.2. 繰返しの各回：新しいデータを登録する。

3.3. 繰返しの各回：関連情報を取得・設定する（処理10：詳細仕様参照）。

1. 処理結果を呼出元へ返す。この経路の後続手順は実行しない。

結果・例外時の扱い

到達条件	結果
この終了経路へ到達した場合	処理結果を呼出元へ返す

sanitizeSheetName

処理概要

外部ライブラリを使わず、単1シートの Open XML (.xlsx) を生成するで「sanitizeSheetName」の要求を受け付ける。入力・確認・データ操作・応答の順と、条件が成立しない場合の処理を以下に示す。

処理手順

1. 関連情報を取得・設定する（処理5：詳細仕様参照）。
2. 関連情報を取得・設定する（処理6：詳細仕様参照）。
3. 「条件7を満たす（原本条件は補足説明を参照）」に該当するか確認する。該当する場合に限り、次の分岐内手順を行う。該当しない場合は分岐の後へ進む。

3.1. 条件成立時：関連情報を取得・設定する（処理8：詳細仕様参照）。

1. 処理結果を呼出元へ返す。この経路の後続手順は実行しない。

結果・例外時の扱い

到達条件	結果
この終了経路へ到達した場合	処理結果を呼出元へ返す

validXmlText

処理概要

外部ライブラリを使わず、単1シートの Open XML (.xlsx) を生成するで「validXmlText」の要求を受け付ける。入力・確認・データ操作・応答の順と、条件が成立しない場合の処理を以下に示す。

処理手順

1. 関連情報を取得・設定する（処理10：詳細仕様参照）。
2. 関連情報を取得・設定する（処理11：詳細仕様参照）。
3. 繰返し条件「条件12を満たす（原本条件は補足説明を参照）」に従い、次の手順を繰り返す。条件を満たさなくなった場合は繰返しの後へ進む。

3.1. 繰返しの各回：関連情報を取得・設定する（処理13：詳細仕様参照）。

3.2. 繰返しの各回：「codePoint が等しい 0x9、またはcodePoint が等しい 0xA、またはcodePoint が等しい 0xD、または条件14を満たす（原本条件は補足説明を参照）、または条件15を満たす（原本条件は補足説明を参照）、または条件16を満たす（原本条件は補足説明を参照）」に該当するか確認する。該当する場合に限り、次の分岐内手順を行う。該当しない場合は分岐の後へ進む。

3.2.1. 条件成立時：関連情報を取得・設定する（処理17：詳細仕様参照）。

3.3. 繰返しの各回：関連情報を取得・設定する（処理18：詳細仕様参照）。

1. 処理結果を呼出元へ返す。この経路の後続手順は実行しない。

結果・例外時の扱い

到達条件	結果
この終了経路へ到達した場合	処理結果を呼出元へ返す

xml

処理概要

外部ライブラリを使わず、単1シートの Open XML (.xlsx) を生成するで「xml」の要求を受け付ける。入力・確認・データ操作・応答の順と、条件が成立しない場合の処理を以下に示す。

処理手順

1. 処理結果を呼出元へ返す。この経路の後続手順は実行しない。

結果・例外時の扱い

到達条件	結果
この終了経路へ到達した場合	処理結果を呼出元へ返す

write

処理概要

外部ライブラリを使わず、単1シートの Open XML (.xlsx) を生成するで「write」の要求を受け付ける。入力・確認・データ操作・応答の順と、条件が成立しない場合の処理を以下に示す。

処理手順

1. 関連情報を取得・設定する（処理4：詳細仕様参照）。
2. 「対象項目」を設定する。
3. 関連情報を取得・設定する（処理5：詳細仕様参照）。
4. 指定した条件でデータを照会する。
5. 関連情報を取得・設定する（処理6：詳細仕様参照）。

補足説明

適用条件

各仮定のQA番号を確認する。静的に確認できた操作・条件を適用する。

処理・条件の原本対応

文書内の条件番号・処理番号は次の原本表記に対応する。完全な業務用語への置換を行っていない箇所はQA・reportで確認する。

イベント	参照番号	原本条件・処理
buildWorkbook	処理1	zip.finish()
buildWorkbook	処理2	write(zip, "xl/worksheets/sheet1.xml", sheetXml(headers, rows, leadingRows, mergedRanges, printScale))
buildWorkbook	処理3	write(zip, "xl/styles.xml", stylesXml())
buildWorkbook	処理4	write(zip, "xl/_rels/workbook.xml.rels", workbookRelationships())
buildWorkbook	処理5	write(zip, "xl/workbook.xml", workbookXml(sanitizeSheetName(sheetName)))
buildWorkbook	処理6	write(zip, "_rels/.rels", rootRelationships())
buildWorkbook	処理7	write(zip, "[Content_Types].xml", contentTypes())
buildWorkbook	処理8	ZipOutputStream zip = new ZipOutputStream(output)
buildWorkbook	処理9	ByteArrayOutputStream output = new ByteArrayOutputStream()
buildWorkbook	処理10	ByteArrayOutputStream output = new ByteArrayOutputStream()
buildWorkbook	処理11	ZipOutputStream zip = new ZipOutputStream(output)
buildWorkbook	処理12	write(zip, "[Content_Types].xml", contentTypes())
buildWorkbook	処理13	write(zip, "_rels/.rels", rootRelationships())
buildWorkbook	処理14	write(zip, "xl/workbook.xml", workbookXml(sanitizeSheetName(sheetName)))
buildWorkbook	処理15	write(zip, "xl/_rels/workbook.xml.rels", workbookRelationships())
buildWorkbook	処理16	write(zip, "xl/styles.xml", stylesXml())
buildWorkbook	処理17	write(zip, "xl/worksheets/sheet1.xml", sheetXml(headers, rows, leadingRows, mergedRanges, printScale))

イベント	参照番号	原本条件・処理
buildWorkbook	処理18	zip.finish()
buildWorkbook	条件19	IOException e
sheetXml	処理1	xml.append("<printOptions horizontalCentered="1"/ >") .append("<pageMargins left="0" right="0" top="0.3937" bottom="0.3937" header="0.1181" footer="0.1181"/ >") .append("<pageSetup paperSize="9" orientation="portrait" scale="") .append(Math.max(10, Math.min(400, printScale))).append(""/>") .append("</worksheet>")
sheetXml	処理2	xml.append("</mergeCells>")
sheetXml	条件3	int i = 0; i < mergedRanges.size(); i++
sheetXml	処理4	xml.append("<mergeCell ref="").append(xml(mergedRanges.get(i))).append(""/>")
sheetXml	処理5	xml.append("<mergeCells count="").append(mergedRanges.size()).append(">")
sheetXml	処理6	xml.append("</sheetData><autoFilter ref="A").append(headerRow).append(':') .append(lastColumn).ap pend(lastRow).append(""/>")
sheetXml	条件7	int rowIndex = 0; rowIndex < rows.size(); rowIndex++
sheetXml	処理8	xml.append("</row>")
sheetXml	条件9	int column = 0; column < headers.size(); column++
sheetXml	処理10	appendNumberCell(xml, reference, (Number) value)
sheetXml	処理11	appendTextCell(xml, reference, value == null ? "" : String.valueOf(value), 0)

イベント	参照番号	原本条件・処理
sheetXml	条件12	value instanceof Number
sheetXml	処理13	String reference = columnName(column + 1) + excelRow
sheetXml	処理14	Object value = row != null && column < row.length ? row[column] : null
sheetXml	処理15	xml.append("<row r="").append(excelRow).append(">")
sheetXml	処理16	Object[] row = rows.get(rowIndex)
sheetXml	処理17	int excelRow = rowIndex + headerRow + 1
sheetXml	処理18	xml.append("</row>")
sheetXml	条件19	int column = 0; column < headers.size(); column++
sheetXml	処理20	appendTextCell(xml, columnName(column + 1) + headerRow, headers.get(column), 1)
sheetXml	処理21	xml.append("<row r="").append(headerRow) .append("" ht="45" customHeight="1">")
sheetXml	条件22	int rowIndex = 0; rowIndex < leadingRows.size(); rowIndex++
sheetXml	処理23	xml.append("</row>")
sheetXml	条件24	int column = 0; column < Math.min(headers.size(), row.length); column++
sheetXml	処理25	appendTextCell(xml, columnName(column + 1) + excelRow, String.valueOf(value), excelRow == 1 ? 3 : 4)
sheetXml	処理26	Object value = row[column]
sheetXml	処理27	xml.append('>')

イベント	参照番号	原本条件・処理
sheetXml	処理28	xml.append(" ht="28" customHeight="1"")
sheetXml	処理29	xml.append("<row r="").append(excelRow).append(""")
sheetXml	処理30	Object[] row = leadingRows.get(rowIndex)
sheetXml	処理31	int excelRow = rowIndex + 1
sheetXml	処理32	xml.append("<sheetData>")
sheetXml	処理33	appendColumns(xml, headers, rows)
sheetXml	処理34	xml.append("<sheetViews><sheetView workbookViewId="0"><pane ySplit="") .append(headerRow).append("" topLeftCell="A").append(headerRow + 1) .append("" activePane="bottomLeft" state="frozen"/></sheetView></ sheetViews>")
sheetXml	処理35	xml.append("<dimension ref="A1:").append(lastColumn).append(lastRow).append(""/>")
sheetXml	処理36	xml.append("<worksheet xmlns="").append(SPREADSHEET_NS).append(">")
sheetXml	処理37	xml.append("<?xml version="1.0" encoding="UTF-8" standalone="yes"?>")
sheetXml	処理38	String lastColumn = columnName(headers.size())
sheetXml	処理39	int lastRow = rows.size() + headerRow
sheetXml	処理40	int headerRow = leadingCount + 1
sheetXml	処理41	int leadingCount = leadingRows == null ? 0 : leadingRows.size()

イベント	参照番号	原本条件・処理
sheetXml	処理42	StringBuilder xml = new StringBuilder(4096 + rows.size() * headers.size() * 40)
sheetXml	処理43	StringBuilder xml = new StringBuilder(4096 + rows.size() * headers.size() * 40)
sheetXml	処理44	int leadingCount = leadingRows == null ? 0 : leadingRows.size()
sheetXml	処理45	int headerRow = leadingCount + 1
sheetXml	処理46	int lastRow = rows.size() + headerRow
sheetXml	処理47	String lastColumn = columnName(headers.size())
sheetXml	処理48	xml.append("<?xml version="1.0" encoding="UTF-8" standalone="yes"?>")
sheetXml	処理49	xml.append("<worksheet xmlns="").append(SPREADSHEET_NS).append(">")
sheetXml	処理50	xml.append("<dimension ref="A1:").append(lastColumn).append(lastRow).append("/>")
sheetXml	処理51	xml.append("<sheetViews><sheetView workbookViewId="0"><pane ySplit="") .append(headerRow).append("" topLeftCell="A").append(headerRow + 1) .append("" activePane="bottomLeft" state="frozen"/></sheetView></ sheetViews>")
sheetXml	処理52	appendColumns(xml, headers, rows)
sheetXml	処理53	xml.append("<sheetData>")
sheetXml	条件54	int rowIndex = 0; rowIndex < leadingRows.size(); rowIndex++

イベント	参照番号	原本条件・処理
sheetXml	処理55	int excelRow = rowIndex + 1
sheetXml	処理56	Object[] row = leadingRows.get(rowIndex)
sheetXml	処理57	xml.append("<row r="").append(excelRow).append(""")
sheetXml	処理58	xml.append(" ht="28" customHeight="1"")
sheetXml	処理59	xml.append('>')
sheetXml	条件60	int column = 0; column < Math.min(headers.size(), row.length); column++
sheetXml	処理61	Object value = row[column]
sheetXml	処理62	appendTextCell(xml, columnName(column + 1) + excelRow, String.valueOf(value), excelRow == 1 ? 3 : 4)
sheetXml	処理63	xml.append("</row>")
sheetXml	処理64	xml.append("<row r="").append(headerRow) .append("" ht="45" customHeight="1">")
sheetXml	条件65	int column = 0; column < headers.size(); column++
sheetXml	処理66	appendTextCell(xml, columnName(column + 1) + headerRow, headers.get(column), 1)
sheetXml	処理67	xml.append("</row>")
sheetXml	条件68	int rowIndex = 0; rowIndex < rows.size(); rowIndex++
sheetXml	処理69	int excelRow = rowIndex + headerRow + 1
sheetXml	処理70	Object[] row = rows.get(rowIndex)
sheetXml	処理71	xml.append("<row r="").append(excelRow).append(">")

イベント	参照番号	原本条件・処理
sheetXml	条件72	int column = 0; column < headers.size(); column++
sheetXml	処理73	Object value = row != null && column < row.length ? row[column] : null
sheetXml	処理74	String reference = columnName(column + 1) + excelRow
sheetXml	条件75	value instanceof Number
sheetXml	処理76	appendNumberCell(xml, reference, (Number) value)
sheetXml	処理77	appendTextCell(xml, reference, value == null ? "" : String.valueOf(value), 0)
sheetXml	処理78	xml.append("</row>")
sheetXml	処理79	xml.append("</sheetData><autoFilter ref="A").append(headerRow).append(':') .append(lastColumn).append(lastRow).append(""/>")
sheetXml	処理80	xml.append("<mergeCells count="").append(mergedRanges.size()).append(">")
sheetXml	条件81	int i = 0; i < mergedRanges.size(); i++
sheetXml	処理82	xml.append("<mergeCell ref="").append(xml(mergedRanges.get(i))).append(""/>")
sheetXml	処理83	xml.append("</mergeCells>")
sheetXml	処理84	xml.append("<printOptions horizontalCentered="1"/>") .append("<pageMargins left="0" right="0" top="0.3937" bottom="0.3937" header="0.1181" footer="0.1181"/>") .append("<pageSetup paperSize="9" orientation="portrait" scale="") .append(Math.max(10, Math.min(400, printScale))).append(""/>") .append("</worksheet>")

イベント	参照番号	原本条件・処理
appendColumns	処理1	xml.append("</cols>")
appendColumns	条件2	int column = 0; column < headers.size(); column++
appendColumns	処理3	xml.append("<col min="").append(column + 1).append("" max="") .append(column + 1).append("" width="").append(width) .append("" customWidth="1"/>")
appendColumns	処理4	width = Math.max(5, Math.min(40, width))
appendColumns	条件5	int row = 0; row < inspectCount; row++
appendColumns	処理6	width = Math.max(width, displayLength(value == null ? "" : String.valueOf(value)) + 2)
appendColumns	処理7	Object value = values != null && column < values.length ? values[column] : null
appendColumns	処理8	Object[] values = rows.get(row)
appendColumns	処理9	int inspectCount = Math.min(rows.size(), 100)
appendColumns	処理10	int width = displayLength(headers.get(column)) + 2
appendColumns	処理11	xml.append("<cols>")
appendColumns	処理12	xml.append("<cols>")
appendColumns	条件13	int column = 0; column < headers.size(); column++
appendColumns	処理14	int width = displayLength(headers.get(column)) + 2
appendColumns	処理15	int inspectCount = Math.min(rows.size(), 100)
appendColumns	条件16	int row = 0; row < inspectCount; row++
appendColumns	処理17	Object[] values = rows.get(row)

イベント	参照番号	原本条件・処理
appendColumns	処理18	Object value = values != null && column < values.length ? values[column] : null
appendColumns	処理19	width = Math.max(width, displayLength(value == null ? "" : String.valueOf(value)) + 2)
appendColumns	処理20	width = Math.max(5, Math.min(40, width))
appendColumns	処理21	xml.append("<col min="").append(column + 1).append("" max="") .append(column + 1).append("" width="").append(width) .append("" customWidth="1"/>")
appendColumns	処理22	xml.append("</cols>")
appendTextCell	処理1	xml.append(">").append(xml(clean)).append("</t></is></c>")
appendTextCell	処理2	xml.append(" xml:space="preserve"")
appendTextCell	条件3	clean.length() > 0
appendTextCell	条件4	(Character.isWhitespace(clean.charAt(0)) || Character.isWhitespace(clean.charAt(clean.length() - 1)))
appendTextCell	処理5	xml.append("<c r="").append(reference).append("" s="").append(style) .append("" t="inlineStr"><is><t")
appendTextCell	処理6	String clean = validXmlText(value)
appendTextCell	処理7	String clean = validXmlText(value)
appendTextCell	処理8	xml.append("<c r="").append(reference).append("" s="").append(style) .append("" t="inlineStr"><is><t")
appendTextCell	条件9	clean.length() > 0
appendTextCell	条件10	(Character.isWhitespace(clean.charAt(0)) || Character.isWhitespace(clean.charAt(clean.length() - 1)))

イベント	参照番号	原本条件・処理
appendTextCell	処理11	xml.append(" xml:space="preserve"")
appendTextCell	処理12	xml.append(">").append(xml(clean)).append("</t></is></c>")
appendNumberCell	処理1	xml.append("<c r="").append(reference).append("" s="2"><v>") .append(String.format(Locale.ROOT, "%s", value)).append("</v></c>")
appendNumberCell	処理2	xml.append("<c r="").append(reference).append("" s="2"><v>") .append(String.format(Locale.ROOT, "%s", value)).append("</v></c>")
displayLength	条件1	int offset = 0; offset < value.length();
displayLength	処理2	offset += Character.charCount(codePoint)
displayLength	処理3	length += codePoint <= 0x7f ? 1 : 2
displayLength	処理4	int codePoint = value.codePointAt(offset)
displayLength	処理5	int length = 0
displayLength	処理6	int length = 0
displayLength	条件7	int offset = 0; offset < value.length();
displayLength	処理8	int codePoint = value.codePointAt(offset)
displayLength	処理9	length += codePoint <= 0x7f ? 1 : 2
displayLength	処理10	offset += Character.charCount(codePoint)
columnName	条件1	value > 0
columnName	処理2	value /= 26
columnName	処理3	value--

イベント	参照番号	原本条件・処理
columnName	処理4	int value = column
columnName	処理5	StringBuilder name = new StringBuilder()
columnName	処理6	StringBuilder name = new StringBuilder()
columnName	処理7	int value = column
columnName	条件8	value > 0
columnName	処理9	value--
columnName	処理10	value /= 26
sanitizeSheetName	処理1	name = name.substring(0, 31)
sanitizeSheetName	条件2	name.length() > 31
sanitizeSheetName	処理3	name = name.replaceAll("[\\/\?*\[\]:]", "_")
sanitizeSheetName	処理4	String name = value == null || value.trim().length() == 0 ? "Sheet1" : value.trim()
sanitizeSheetName	処理5	String name = value == null || value.trim().length() == 0 ? "Sheet1" : value.trim()
sanitizeSheetName	処理6	name = name.replaceAll("[\\/\?*\[\]:]", "_")
sanitizeSheetName	条件7	name.length() > 31
sanitizeSheetName	処理8	name = name.substring(0, 31)
validXmlText	条件1	int offset = 0; offset < source.length();
validXmlText	処理2	offset += Character.charCount(codePoint)
validXmlText	処理3	result.appendCodePoint(codePoint)

イベント	参照番号	原本条件・処理
validXmlText	条件4	(codePoint >= 0x20 && codePoint <= 0xD7FF)
validXmlText	条件5	(codePoint >= 0xE000 && codePoint <= 0xFFFD)
validXmlText	条件6	(codePoint >= 0x10000 && codePoint <= 0x10FFFF)
validXmlText	処理7	int codePoint = source.codePointAt(offset)
validXmlText	処理8	String source = value == null ? "" : value
validXmlText	処理9	StringBuilder result = new StringBuilder(value == null ? 0 : value.length())
validXmlText	処理10	StringBuilder result = new StringBuilder(value == null ? 0 : value.length())
validXmlText	処理11	String source = value == null ? "" : value
validXmlText	条件12	int offset = 0; offset < source.length();
validXmlText	処理13	int codePoint = source.codePointAt(offset)
validXmlText	条件14	(codePoint >= 0x20 && codePoint <= 0xD7FF)
validXmlText	条件15	(codePoint >= 0xE000 && codePoint <= 0xFFFD)
validXmlText	条件16	(codePoint >= 0x10000 && codePoint <= 0x10FFFF)
validXmlText	処理17	result.appendCodePoint(codePoint)
validXmlText	処理18	offset += Character.charCount(codePoint)
write	処理1	zip.closeEntry()
write	処理2	zip.putNextEntry(entry)
write	処理3	ZipEntry entry = new ZipEntry(path)

イベント	参照番号	原本条件・処理
write	処理4	ZipEntry entry = new ZipEntry(path)
write	処理5	zip.putNextEntry(entry)
write	処理6	zip.closeEntry()